

Disciplina: Aprendizado Profundo

Aula 8: Laboratório 3 - CNNs, PyTorch e Hugging Face

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

8 de Outubro de 2025

Roteiro da Aula de Hoje

- 1 Revisão e Motivação
- 2 Tutorial Rápido: PyTorch e Hugging Face
- 3 Exercícios Práticos no Colab
- 4 Próximos Passos

Recapitulando a Aula 7

Os Pilares das CNNs

- **Viés Indutivo:** CNNs são projetadas com suposições sobre dados espaciais: localidade, compartilhamento de pesos e hierarquia.
- **Operação de Convolução:** Um kernel (filtro) aprende a detectar um padrão e é deslizado sobre a imagem para criar um mapa de features.
- **Pooling:** Uma operação de downsampling que reduz a dimensionalidade e confere invariância local a pequenas translações.
- **Aprendizado Hierárquico:** Camadas empilhadas aprendem features de complexidade crescente.

Objetivo do Laboratório: CNNs como Extratores de Features

Além da Classificação

Uma CNN treinada para classificação aprende a mapear uma imagem para uma representação de baixa dimensão (um vetor de features, ou **embedding**) que é semanticamente rica. E se ignorarmos a última camada e usarmos o vetor de features diretamente?

A Nova Tarefa: Busca de Imagens Baseada em Conteúdo (CBIR)

O objetivo é: dada uma imagem de consulta, encontrar as imagens mais **visualmente similares** em um grande banco de dados, medindo a “distância” entre seus vetores de features.

PyTorch: Tensores

O que é um Tensor?

Um `'torch.Tensor'` é um array n-dimensional, a estrutura de dados fundamental do PyTorch. É similar ao `'numpy.ndarray'`, mas com duas superpotências:

- 1 Aceleração por GPU.
- 2 Rastreamento de operações para diferenciação automática (Autograd).

PyTorch: Tensores

```
x = torch.tensor([[1., 2.], [3., 4.]], requires_grad=True)
y = x + 2
z = y * y * 3
out = z.mean()
# out é um escalar que depende de x
```

PyTorch: Tensores

Operações Úteis

```
# Criando tensores
```

```
x = torch.randn(3, 4) # Matriz 3x4 aleatória
```

```
y = torch.ones_like(x) # Matriz de 1s com o mesmo shape de x
```

```
# Manipulando o shape (muito importante!)
```

```
z = x.view(12) # Achata para um vetor de 12 elementos
```

```
z = x.view(-1, 2) # Remodela para (6, 2), -1 infere a dimensão
```

```
# Operações
```

```
w = torch.matmul(x.T, x) # Multiplicação de matrizes
```

PyTorch: Tensores

Parâmetros Aprendíveis

Para os pesos de uma rede, usamos `'torch.nn.Parameter'`, uma subclasse de `Tensor` que automaticamente se registra como um parâmetro do modelo quando atribuído a um `'nn.Module'`.

Desafio: Tensor Puzzles

Quebra-cabeça 1: Softmax Estável

Implemente uma função que receba um tensor de logits de forma '(B, C)' e retorne as probabilidades softmax. Sua implementação deve ser numericamente estável.

```
def stable_softmax(logits):  
    # Dica: subtraia o valor máximo de cada linha  
    # antes de aplicar a exponencial para evitar overflow.  
    ...  
    return probs
```

Desafio: Tensor Puzzles

Quebra-cabeça 2: One-Hot Encoding

Implemente uma função que receba um tensor de rótulos '(B,)' e o número de classes 'C', e retorne uma matriz one-hot de forma '(B, C)'.

```
def one_hot(labels, num_classes):  
    # Dica: use torch.zeros e o método .scatter_()  
    ...  
    return one_hot_matrix
```

PyTorch: Autograd, o Motor de Gradientes

Cálculo Automático de Gradientes

Quando `'requires_grad=True'`, o PyTorch constrói um grafo computacional dinâmico. Chamar `'.backward()'` em um escalar (como o custo) percorre o grafo ao contrário e acumula os gradientes.

Exercício Rápido

Considere a função $y = \sum_i \log(x_i^2)$. Calcule $\frac{dy}{dx}$ manualmente para $x = [1, 2]$. Depois, verifique seu resultado com o código abaixo.

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = torch.log(x.pow(2)).sum()
y.backward()
print(x.grad) # 0 que você espera que seja impresso?
# Resposta:  $dy/dx_i = 2/x_i \rightarrow [2.0, 1.0]$ 
```

Mergulho Raso: Autograd para Simular Sistemas Dinâmicos

Resolvendo uma ODE com Autograd

Podemos usar o Autograd para resolver problemas que vão além do treinamento de redes. Exemplo: encontrar o parâmetro de um sistema dinâmico.

Considere a ODE de decaimento: $\frac{dy}{dt} = -k \cdot y(t)$. O método de Euler para simular um passo é: $y_{t+1} = y_t + \Delta t \cdot (-k \cdot y_t)$.

Mergulho Raso: Autograd para Simular Sistemas Dinâmicos

Aprendendo o Parâmetro k

Podemos tratar k como um parâmetro aprendível e usar o Autograd para encontrar o valor que melhor se ajusta a dados observados.

```
# Parâmetro que queremos aprender
k = torch.tensor([0.5], requires_grad=True)
# Simulação com Euler (forward pass)
y_sim = [torch.tensor([10.0])] # Condição inicial
for t in range(10):
    y_next = y_sim[-1] - k * y_sim[-1] * 0.1 # dt = 0.1
    y_sim.append(y_next)
loss = torch.sum((torch.stack(y_sim) - y_real_data)**2) # Custo
# O Autograd faz o resto!
# Agora k.grad contém a direção para ajustar k
loss.backward()
```

PyTorch: Construindo Modelos com 'nn.Module'

Definindo uma Rede

Todos os modelos em PyTorch devem herdar da classe 'nn.Module'.

- No '.__init__.', definimos as camadas que usaremos (e.g., 'nn.Conv2d', 'nn.Linear').
- No 'forward', definimos como os dados fluem através dessas camadas.

PyTorch: Construindo Modelos com 'nn.Module'

```
class SimpleCNN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 16, kernel_size=5)  
        self.relu = nn.ReLU()  
        self.pool = nn.MaxPool2d(2, 2)  
        self.fc1 = nn.Linear(16 * 12 * 12, 10)  
  
    def forward(self, x):  
        x = self.pool(self.relu(self.conv1(x)))  
        x = x.view(x.size(0), -1) # Flatten  
        x = self.fc1(x)  
        return x
```

Ecosistema Hugging Face: Simplificando o Acesso

O que é o Hugging Face?

É uma plataforma (e um conjunto de bibliotecas) que democratiza o acesso a modelos pré-treinados e datasets.

- **Hub de Modelos:** Um repositório com milhares de modelos para NLP, Visão Computacional, etc.
- **Biblioteca 'transformers':** Carregue qualquer modelo do Hub com uma única linha de código.
- **Biblioteca 'datasets':** Carregue e processe datasets padronizados de forma eficiente.

Ecosistema Hugging Face: Simplificando o Acesso

Exemplo Prático

```
from transformers import AutoModelForImageClassification
from datasets import load_dataset

# Carregar um modelo ResNet-18 pré-treinado no ImageNet
model = AutoModelForImageClassification.from_pretrained(
    "microsoft/resnet-18"
)

# Carregar o dataset CIFAR-10
dataset = load_dataset("cifar10")
```

Exercício 1: Usando um Modelo Pré-treinado

A Tarefa

No notebook, use a biblioteca 'transformers' para carregar um modelo de CNN (e.g., LeNet-5) pré-treinado no dataset Fashion-MNIST.

Exercício 2: Visualizando os Kernels

O que a Rede Aprendeu?

Extraia os pesos (kernels) da **primeira camada convolucional** do modelo pré-treinado e visualize-os como imagens.

O que esperamos ver?

Os filtros devem se parecer com detectores de padrões simples e genéricos, como bordas horizontais, verticais, diagonais e gradientes de cor. Estas são as "letras" do alfabeto visual que a rede aprendeu.

Exercício 3: Criando um Modelo Menor com Kernels Extraídos

Transferindo o Conhecimento

Crie um novo modelo PyTorch que consiste em **apenas uma camada convolucional**. Em vez de inicializar os pesos aleatoriamente, **copie os pesos** da primeira camada do modelo pré-treinado.

Tarefa

Passe uma imagem do Fashion-MNIST através do seu novo modelo de uma camada e visualize os mapas de features resultantes. O que eles representam? (Dica: representam a resposta da imagem a cada um dos detectores de bordas que você visualizou).

Exercício 4: Implementando o Motor de Busca de Imagens

O Algoritmo de Busca (CBIR)

- 1 Extrair Embeddings:** Use o modelo pré-treinado (sem a camada de classificação final) para extrair os vetores de features (embeddings) de todas as imagens do conjunto de teste.
- 2 Consulta:** Escolha uma imagem de consulta e extraia seu embedding.
- 3 Busca:** Calcule a distância Euclidiana entre o embedding da consulta e todos os outros.
- 4 Visualização:** Mostre a imagem de consulta e as 5 imagens com os embeddings mais próximos.

Encerramento e Próximos Passos I

Resumo do Laboratório

- Aprendemos os fundamentos do PyTorch (Tensores, Autograd, 'nn.Module').
- Exploramos como carregar modelos e datasets com o Hugging Face.
- Usamos uma CNN como um extrator de features para construir um motor de busca de imagens.

Encerramento e Próximos Passos II

Para a Próxima Aula

- **Tópico:** Redes Neurais Recorrentes (RNNs).
- **Objetivo:** Mudar do domínio de imagens para o de dados sequenciais (texto, séries temporais) e introduzir a ideia de estado recorrente.

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltpf_book.pdf.
-  Belkin, Mikhail et al. (2019). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. Em: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.

Referências II

-  Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.
-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefter, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.

Referências III

-  Hastie, Trevor et al. (2022). “Surprises in high-dimensional ridgeless least squares interpolation”. Em: *The Annals of Statistics* 50.2, pp. 949–986.
-  Hochreiter, Sepp e Jürgen Schmidhuber (1997). “Flat minima”. Em: *Neural computation* 9.1, pp. 1–42.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Keskar, Nitish Shirish et al. (2017). “On large-batch training for deep learning: Generalization gap and sharp minima”. Em: *International Conference on Learning Representations (ICLR)*.
-  Kingma, Diederik P. e Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. Em: *ICLR 2015*.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências IV

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: [10.1007/bf02478259](https://doi.org/10.1007/bf02478259).

Referências V

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Nesterov, Yurii (2004). *Introductory lectures on convex optimization: A basic course*. Vol. 87. Springer Science & Business Media.

Referências VI

-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. Em: *Journal of mathematical biology* 15.3, pp. 267–273.
-  Paes, Lucas Monteiro et al. (2023). “On the Inevitability of the Rashomon Effect”. Em: *IEEE Journal on Selected Areas in Information Theory* 4.2, pp. 416–427. DOI: 10.1109/JSAIT.2023.3287661.
-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.”. Em: *Psychological review* 65.6, p. 386.

Referências VII

-  Rudin, Cynthia et al. (2024). “Position: amazing things come from having many good models”. Em: *ICML*.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.
-  Shannon, C. E. (1988). “Programming a computer for playing chess”. Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.

Referências VIII

-  Turing, Alan M. (1969). “Intelligent Machinery”. Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.
-  Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).